



Computer Networks

Week 5 — Detailed Study Notes

Flow Control · Congestion Control: Signals, Knobs & the Steady State · ACK Clocking · Slow Start · Congestion Avoidance (TCP New Reno Additive Increase)

IIT Madras BS — Prof. Ayon Chakraborty

How to read these notes. Every slide from the three Week-5 lectures appears here, in slide order, grouped by module (Module 5. n = Lecture n), with the professor's own figures embedded next to the matching text. Box types: **Definition**, **Intuition**, **Formula** (derivations + [units]), **Common pitfall**, **Beyond the slides** (flagged extra material), grey **What the slide says**. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 5.1 — Flow Control	2
1.1	Review: the app and the socket buffer	2
1.2	But socket buffers can get full.	2
1.3	Goal: avoid drops due to buffer fill	3
1.4	Flow control in TCP headers	3
1.5	TCP flow control in action	4
1.6	Sizing the receiver's socket buffer	5
1.7	Summary of flow control	6
2	Module 5.2 — Congestion Control and the Steady State	6
2.1	Congestion control: the Internet's approach	6
2.2	Signals and knobs	6
2.3	Sense and react — but how?	7
2.4	The steady state: efficiency of a single TCP conversation	7
2.5	When to send the next packet? ACK clocking	8
2.6	ACK clocking alone can be inefficient	9
2.7	Steady state of congestion control	10
3	Module 5.3 — Finding the Right Congestion Window	10
3.1	The problem: a number-guessing game	10
3.2	Quickly finding a rate: TCP slow start	11
3.3	Behaviour of slow start alone	12
3.4	TCP congestion avoidance: New Reno's additive increase	13
3.5	Behaviour of additive increase (the slide's example)	14
3.6	The zoo of TCP variants	15
4	Formula Sheet	16
5	Numerical Problems with Worked Solutions	16
6	Multiple-Choice Questions with Explanations	18

1 Module 5.1 — Flow Control

1.1 Review: the app and the socket buffer

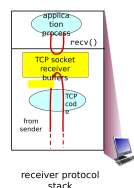
What the slide says (W5 L1, slide 2)

The sender deposits data in the receiver socket buffer. An app with a TCP socket reads from the TCP receive socket buffer (e.g. `data = sock.recv()`). Buffers are used for **ordering** and **reliability**. **Ordering**: only release data to the app when it is in order with everything else the app has read previously. **Reliability**: avoid wasteful sender retransmissions using selective repeat.

From the lecture slides: W5 L1, slides 2–4 — the receiver protocol stack; buffers can fill; goal: avoid drops due to buffer fill

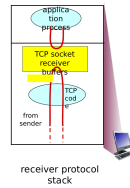
Review: app and socket buffer interaction

- Sender deposits data in receiver socket buffer
- An app with a TCP socket reads from the TCP receive socket buffer
 - e.g., when you do `data = sock.recv()`
- Buffers used for ordering & reliability
- Ordering: only release data to app when data **in order** with everything else app has read previously
- Reliability: avoid wasteful sender retransmissions using selective repeat



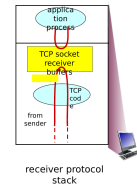
But socket buffers can get full...

- Applications may read data slower than the sender is pushing data in
 - Example: what if an app infrequently or never calls `recv()`?
- There may be too much reordering or packet loss in the network
 - What if the first few bytes of a window are lost or delayed?
- Receivers can only buffer so much before dropping subsequent data



Goal: avoid drops due to buffer fill

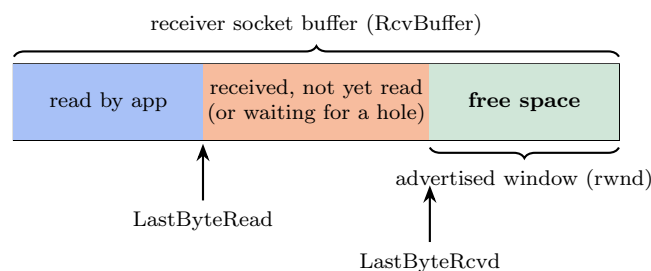
- Have a TCP sender only send as much as the **free buffer space** available at the receiver.
- Amount of free buffer varies over time
- TCP implements **flow control**
- Receiver's ACK contains the amount of data the sender can transmit without running out the receiver's socket buffer
- This number is called the **advertised window size**



1.2 But socket buffers can get full...

What the slide says (W5 L1, slide 3)

- Applications may read data slower than the sender is pushing it in — e.g. what if an app infrequently or never calls `recv()`?
- There may be too much reordering or packet loss in the network — what if the first few bytes of a window are lost or delayed?
- Receivers can only buffer so much before dropping subsequent data.



Intuition: Two ways the buffer fills

(1) A **slow reader**: data arrives in order, but the app doesn't call `recv()` often enough, so the "received, not read" region grows. (2) A **hole**: the first segment of a window is lost; everything after it must wait in the buffer (TCP reassembly, Week 4) even if the app is eager to read. Either way, once the buffer is full, newly arriving segments are *dropped at the receiver* — even though they crossed the whole network successfully. That wastes network capacity and forces retransmissions.

1.3 Goal: avoid drops due to buffer fill

What the slide says (W5 L1, slide 4)

Have a TCP sender only send as much as the **free buffer space** available at the receiver. The amount of free buffer varies over time. TCP implements **flow control**: the receiver's ACK contains the amount of data the sender can transmit without running out the receiver's socket buffer. This number is called the **advertised window size**.

Definition: Flow control

A mechanism that prevents the sender from overflowing the *receiver's* buffer, by limiting the sender's unACKed data to the free space the receiver advertises.

Definition: Advertised (receive) window, rwnd

The number of bytes of free space in the receiver's socket buffer, carried in the 16-bit **Window** field of every TCP segment the receiver sends (i.e. in the ACKs).

Formula: Computing the advertised window

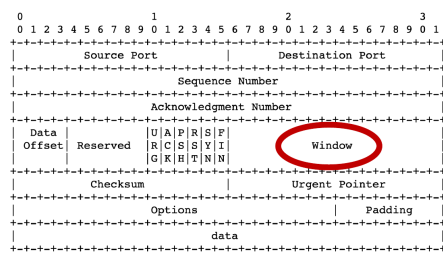
$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead}) \quad [\text{bytes}]$$

Derivation. Bytes currently occupying the buffer are those received but not yet read by the app: $\text{LastByteRcvd} - \text{LastByteRead}$. Whatever remains of the buffer is free, and that is exactly how much more the sender may send. (Standard Kurose & Ross notation; the slides state the idea in words.)

1.4 Flow control in TCP headers

From the lecture slides: W5 L1, slides 5–6 — the Window field of the TCP header carries the advertised window

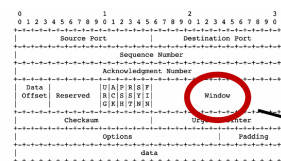
Flow control in TCP headers



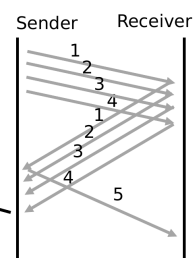
Note that one tick mark represents one bit position.

TCP flow control

- Receiver **advertises** to sender (in the ACK) how much free buffer is available



Note that one tick mark represents one bit position.

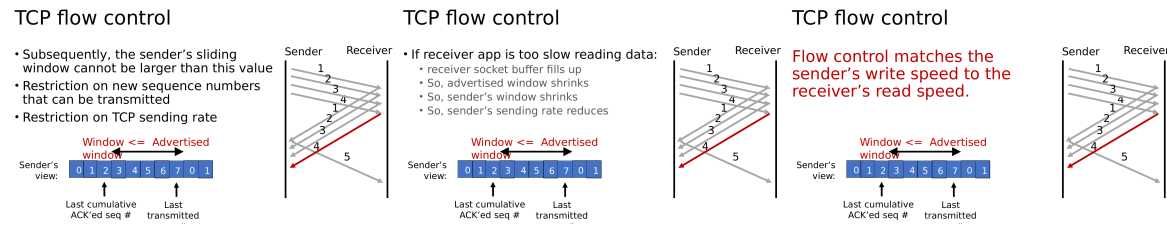


Common pitfall: 16 bits = 64 KB?

The Window field is 16 bits, so without extensions $\text{rwnd} \leq 65,535$ bytes. For fast long paths this is too small (see the throughput bound below); TCP's *window scale option* (RFC 7323) multiplies it by up to 2^{14} . (Beyond the slides.)

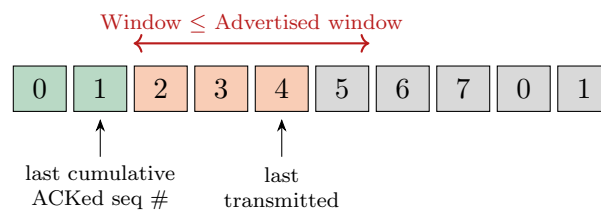
1.5 TCP flow control in action

From the lecture slides: W5 L1, slides 7–9 — the receiver advertises free space; the sender's window cannot exceed it; a slow reader shrinks it



What the slide says (W5 L1, slides 6–9)

- The receiver **advertises** to the sender (in the ACK) how much free buffer is available.
- Subsequently, the sender's sliding window cannot be larger than this value: a restriction on the new sequence numbers that can be transmitted, i.e. a restriction on the TCP sending rate. **Window $\leq$ Advertised window.**
- If the receiver app is too slow reading data: the receiver socket buffer fills up, so the advertised window shrinks, so the sender's window shrinks, so the sender's sending rate reduces.
- Flow control matches the sender's write speed to the receiver's read speed.**



Intuition: A chain of cause and effect

slow app $\rightarrow$ buffer fills $\rightarrow$ rwnd in ACKs shrinks $\rightarrow$ sender's permitted window shrinks $\rightarrow$ fewer bytes in flight per RTT $\rightarrow$ lower sending rate $\rightarrow$ arrival rate $\approx$ app's read rate. The feedback loop settles exactly where the sender writes as fast as the receiver reads — no faster (no drops), no slower (if the reader speeds up, rwnd grows again).

Formula: Throughput bound from flow control

Since at most rwnd bytes can be unACKed and ACKs return once per RTT:

$$\text{Throughput} \leq \frac{\text{rwnd}}{\text{RTT}} \quad \left[\frac{\text{bytes}}{\text{s}} \right]$$

E.g. rwnd = 65,535 B, RTT = 100 ms: $\leq 655 \text{ KB/s} = 5.24 \text{ Mbps}$, no matter how fast the link is.

Beyond the slides: Zero window

If rwnd reaches 0 the sender must stop. To avoid a deadlock (the receiver's "window opened" update could be lost), the sender periodically sends a tiny *zero-window probe*, which elicits a fresh ACK carrying the current rwnd.

1.6 Sizing the receiver's socket buffer

From the lecture slides: W5 L1, slides 10–12 — how big should the receiver socket buffer be? Case 1 and Case 2

Sizing the receiver's socket buffer

- Operating systems have a default receiver socket buffer size

- Listed among the parameters in `sysctl -a | grep net.ipv4.tcp` on Linux

- If socket buffer is too small, sender can't keep too many packets in flight → lower throughput

- If socket buffer is too large, too much memory consumed per socket

- How big should the receiver socket buffer be?

Sizing the receiver's socket buffer

- Case 1: Suppose the receiving app is reading data too slowly:

- no amount of receiver buffer can prevent low sender throughput if the connection is long-lived

Sizing the receiver's socket buffer

- Case 2: Suppose the receiving app reads sufficiently fast on average to match the sender's writing speed.

- Assume the sender has a window of size W .
- The receiver must use a buffer of size at least W .

- Captures two cases:

- (1) When the first sequence #s in the window are dropped
 - The rest of the window must be buffered until the ACKs (of the rest of the window) reach sender. Adv. window in ACKs reduces sender's window

- (2) When the sender sends a burst of data of size W
 - Receiver may not match the *instantaneous* rate of the sender

What the slide says (W5 L1, slides 10–12)

Operating systems have a default receiver socket buffer size (listed among the parameters in `sysctl -a | grep net.ipv4.tcp` on Linux). Too small ⇒ the sender can't keep many packets in flight ⇒ lower throughput. Too large ⇒ too much memory consumed per socket.

Case 1: the receiving app reads too slowly: no amount of receiver buffer can prevent low sender throughput if the connection is long-lived. **Case 2:** the receiving app reads sufficiently fast *on average* to match the sender's writing speed. Assume the sender has a window of size W . **The receiver must use a buffer of size at least W .** This captures two cases: (1) when the first sequence #s in the window are dropped — the rest of the window must be buffered until the ACKs (of the rest of the window) reach the sender; the advertised window in ACKs reduces the sender's window; (2) when the sender sends a burst of size W — the receiver may not match the *instantaneous* rate of the sender.

Intuition: Why “at least W ”

Case 1 is a rate mismatch: if the app reads at 1 MB/s and the sender writes at 10 MB/s forever, any finite buffer fills eventually; after that the flow runs at 1 MB/s. A bigger buffer only delays the inevitable. **Case 2** is a *timing* mismatch: averages match, but (1) a lost first segment forces the receiver to hold the other $W - 1$ segments' worth of data until the retransmission fills the hole, and (2) the sender may emit its whole window as one burst, faster than the app drains it. In both situations up to a full window sits in the buffer at once, so the buffer must be $\geq W$.

Formula: How big should W (and so the buffer) be?

To keep the path full, $W \approx R_{\text{bottleneck}} \times \text{RTT}$ (Week 2's BDP, round-trip version). Hence

$$\text{RcvBuffer} \gtrsim R \times \text{RTT} \quad \left[\text{bits} = \frac{\text{bits}}{\text{s}} \cdot \text{s} \right]$$

Example: $1 \text{ Gbps} \times 20 \text{ ms} = 2 \times 10^7 \text{ bits} = 2.5 \text{ MB}$ per connection.

Beyond the slides: Linux knobs

`net.ipv4.tcp_rmem = min default max` (e.g. 4096 131072 6291456); Linux auto-tunes the buffer between min and max. Try `sysctl net.ipv4.tcp_rmem` on your machine.

1.7 Summary of flow control

Summary: Flow control (slide)

A mechanism to keep buffers available at the receiver whenever the sender transmits data. Main function: **match sender speed to receiver speed**. Socket buffer sizing is important for throughput.

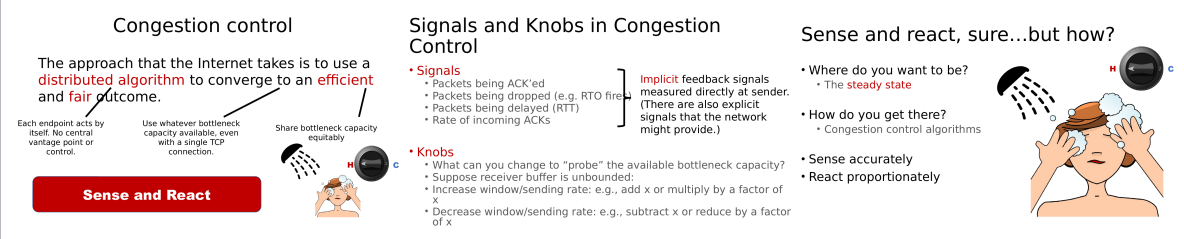
2 Module 5.2 — Congestion Control and the Steady State

2.1 Congestion control: the Internet's approach

What the slide says (W5 L2, slide 2)

The approach that the Internet takes is to use a **distributed algorithm** to converge to an **efficient** and **fair** outcome. *Distributed*: each endpoint acts by itself; no central vantage point or control. *Efficient*: use whatever bottleneck capacity is available, even with a single TCP connection. *Fair*: share bottleneck capacity equitably. $\Rightarrow$ **Sense and React** (the shower-knob picture: too hot, turn toward cold; too cold, turn toward hot).

From the lecture slides: W5 L2, slides 2–4 — distributed, efficient, fair; signals and knobs; sense and react



Intuition: The shower analogy

You can't see the boiler; you only feel the water. Too hot $\rightarrow$ turn toward C; too cold $\rightarrow$ turn toward H. There's a lag between turning the knob and feeling the change, so turning too hard makes you oscillate between scalding and freezing. Congestion control is the same: the bottleneck is remote and invisible, feedback arrives an RTT later, and over-reacting causes oscillation. Hence "sense accurately, react proportionately".

Definition: Congestion control vs flow control

Flow control protects the *receiver's buffer* (explicit number: `rwnd`). **Congestion control** protects the *network* — router buffers and bottleneck links — and the network gives no explicit number, so the sender must infer the right rate from signals.

2.2 Signals and knobs

What the slide says (W5 L2, slide 3)

Signals (implicit feedback measured directly at the sender): packets being ACKed; packets being dropped (e.g. RTO fires); packets being delayed (RTT); rate of incoming ACKs. (There are also explicit signals that the network might provide.) **Knobs**: what can you change

to “probe” the available bottleneck capacity? Suppose the receiver buffer is unbounded. Increase the window/sending rate (e.g. add x , or multiply by a factor of x). Decrease the window/sending rate (e.g. subtract x , or reduce by a factor of x).

Signal	What it suggests
ACKs arriving	data is getting through; there may be room for more
Packet dropped (RTO fires)	a buffer somewhere overflowed: strong congestion signal
RTT increasing	queues are building at the bottleneck: early congestion signal
Rate of incoming ACKs	the rate data crosses the bottleneck (used by BBR, Week 6)
Explicit (e.g. ECN marks)	routers flag congestion before dropping (beyond the slides)

Knob action	Additive	Multiplicative
Increase	$\text{cwnd} \leftarrow \text{cwnd} + x$	$\text{cwnd} \leftarrow \text{cwnd} \times x$
Decrease	$\text{cwnd} \leftarrow \text{cwnd} - x$	$\text{cwnd} \leftarrow \text{cwnd} / x$

Definition: Congestion window, cwnd

The sender’s own limit on unACKed data, set by the congestion-control algorithm (not advertised by anyone). It is the knob.

2.3 Sense and react — but how?

What the slide says (W5 L2, slide 4)

Where do you want to be? **The steady state.** How do you get there? **Congestion control algorithms.** Sense accurately. React proportionately.

2.4 The steady state: efficiency of a single TCP conversation

From the lecture slides: W5 L2, slides 6–7 — efficiency of one TCP conversation; the ideal goal

What does **efficiency** look like?

- Suppose we want to achieve an **efficient** outcome for **one** TCP conversation by observing network signals from the endpoint



- Q: How should the endpoint behave **at steady state**?
- Challenge: bottleneck link is remotely located

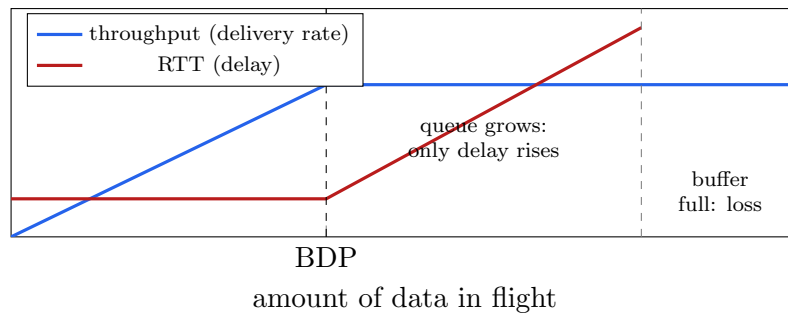
Steady state: Ideal goal

- High sending rate:** Use the full capacity of the bottleneck link
- Low delay:** Minimize the overall delay of packets to get to the receiver
 - Overall delay = propagation + queueing + transmission
 - Assume propagation and transmission components fixed
- “Low delay” reduces to **low queueing delay**
- i.e., don’t push so much data into the network that packets have to wait in queues
- Key question: When to send the next packet?

What the slide says (W5 L2, slides 6–7)

Suppose we want an efficient outcome for one TCP conversation by observing network signals from the endpoint. Q: how should the endpoint behave at steady state? Challenge: the bottleneck link is remotely located. **Ideal goal.** *High sending rate:* use the full capacity of the bottleneck link. *Low delay:* minimise the overall delay of packets to get to the receiver. Overall delay = propagation + queueing + transmission; assume propagation and transmission are fixed. “Low delay” reduces to **low queueing delay**, i.e. don’t push so much data into

the network that packets have to wait in queues. Key question: **when to send the next packet?**

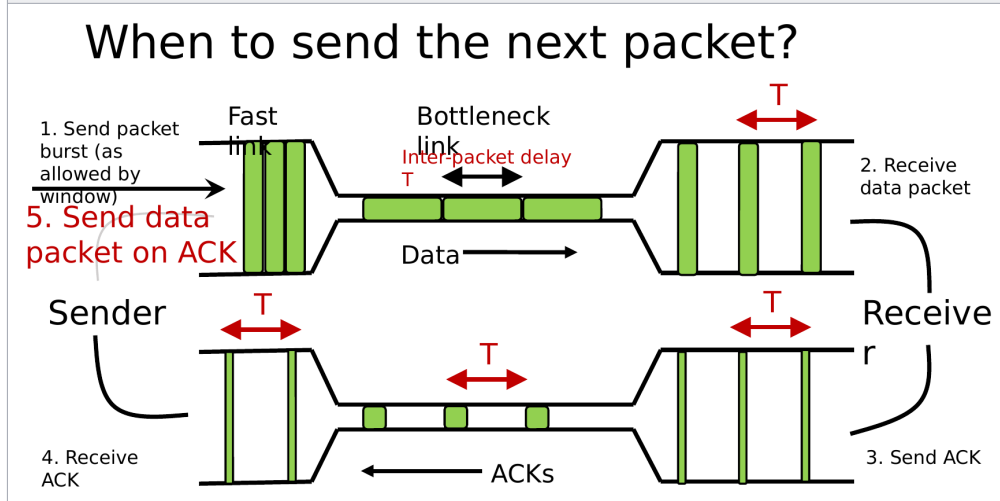


Intuition: The optimal operating point

With less than one BDP in flight, the link idles part of the time. Beyond one BDP, extra data doesn't go faster — it just sits in the bottleneck queue, raising delay. Past the buffer size, it's dropped. The ideal steady state sits exactly at the knee: **in-flight** = **bottleneck rate** × **base RTT** — maximum throughput with minimum delay. (This is exactly the point Google's BBR tries to find, Week 6.)

2.5 When to send the next packet? ACK clocking

From the lecture slides: W5 L2, slide 8 — a burst crosses the bottleneck; its spacing T is preserved in the ACKs; the sender sends one packet per ACK



What the slide says (W5 L2, slide 8)

(1) Send a packet burst (as allowed by the window) on a fast link. (2) At the bottleneck, packets get spread out with inter-packet delay T ; the receiver receives data packets spaced T apart. (3) The receiver sends ACKs — also spaced T . (4) The sender receives ACKs spaced T . (5) The sender sends a data packet on each ACK.

Formula: The spacing T equals the bottleneck transmission time

$$T = \frac{L}{R_{\text{bottleneck}}} \quad [\text{s}]$$

Derivation. The bottleneck can only put one packet of L bits on its wire every L/R_b seconds; packets that arrived back-to-back leave it L/R_b apart. Faster links downstream can't bring them closer (they each still arrive one at a time). The receiver ACKs each on arrival, so ACKs are T apart, and the ACK path preserves that spacing. So the ACK arrival rate $1/T$ packets/s is exactly the bottleneck's packet rate.

From the lecture slides: W5 L2, slides 9–10 — rationale: packet conservation and ACK clocking; the grocery-store analogy

Rationale

- When the sender receives an ACK, that's a signal that the previous packet has left the bottleneck link (and the rest of the network)
- Hence, **it must be safe to send another packet without congesting the bottleneck link**
- Such transmissions are said to follow **packet conservation**
- **ACK clocking**: "Clock" of ACKs governs packet transmissions

ACK clocking: analogy

- How to avoid crowding a grocery store?
- Strategy: Send the next waiting customer exactly when a customer exits the store
- However, this strategy alone can lead to inefficient use of resources...



What the slide says (W5 L2, slide 9)

When the sender receives an ACK, that's a signal that the previous packet has **left the bottleneck link** (and the rest of the network). Hence it must be safe to send another packet without congesting the bottleneck link. Such transmissions follow **packet conservation**. **ACK clocking**: the "clock" of ACKs governs packet transmissions.

Definition: Packet conservation / ACK clocking

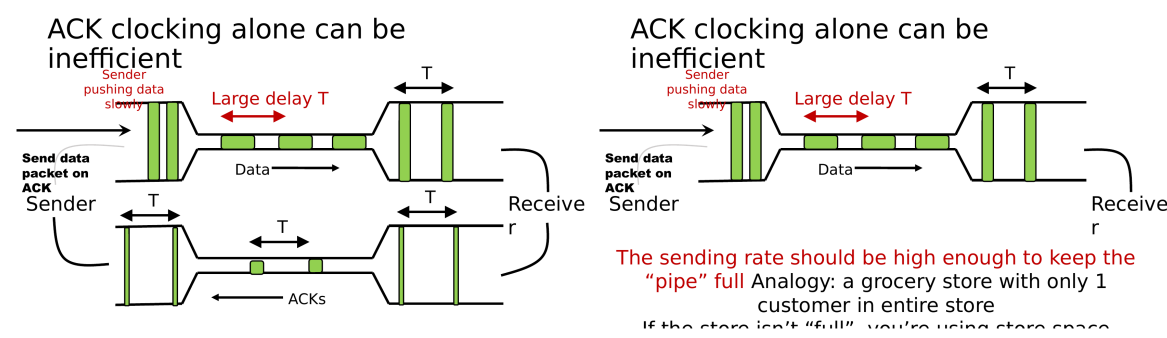
In steady state, put a new packet into the network only when an old one leaves it (signalled by its ACK). The ACK stream acts as a clock ticking at exactly the bottleneck rate, so the sender automatically sends at the bottleneck rate without ever measuring it.

Intuition: The grocery store (slide 10)

To avoid crowding a store, the guard lets the next waiting customer in exactly when one exits. The store never overflows, and the entry rate automatically matches the checkout rate. That's ACK clocking — each ACK is "a customer left".

2.6 ACK clocking alone can be inefficient

From the lecture slides: W5 L2, slides 11–12 — a sender pushing data slowly: large gaps; the pipe is not full



What the slide says (W5 L2, slides 11–12)

If the sender is pushing data slowly (large delay T between its packets), ACK clocking will faithfully reproduce that slow pace: the pipe is never full. The sending rate should be high enough to keep the “pipe” full. Analogy: a grocery store with only 1 customer in the entire store — if the store isn’t full, you’re using store space (and staff) inefficiently.

Intuition

ACK clocking only *preserves* the current number of packets in flight — one new packet per departing packet. It keeps you from making things worse, but never makes things better. If you start with 2 packets in flight on a pipe that holds 50, you’ll stay at 2 forever. Something else must *grow* the amount in flight up to the pipe size: that is the job of slow start and additive increase (Module 5.3).

2.7 Steady state of congestion control

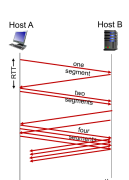
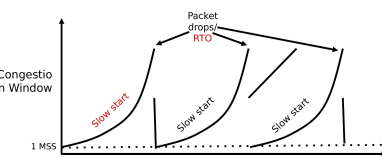
Summary: Steady state (slide)

Send at the **highest rate possible** (to keep the pipe full) **while being ACK-clocked** (to avoid congesting the pipe). So, how to get to steady state? $\Rightarrow$ Module 5.3.

3 Module 5.3 — Finding the Right Congestion Window

3.1 The problem: a number-guessing game

From the lecture slides: W5 L3, slides 2–4 — the guessing game; slow start; behaviour of slow start

Finding the right congestion window	Quickly finding a rate: TCP slow start	Behavior of slow start
- Recall the number guessing game. TCP congestion control algorithms solve a similar problem! - There is an unknown bottleneck link rate that the sender must match • packet loss, delays, etc. - If sender sends more than the bottleneck link rate: • packet loss, delays, etc. - If sender sends less than the bottleneck link rate: • all packets get through; successful ACKs	- Initially $cwnd = 1\text{ MSS}$ • MSS is “maximum segment size” - Upon receiving an ACK of each MSS, increase the $cwnd$ by 1 MSS - Effectively, double $cwnd$ every RTT - Initial rate is slow but ramps up exponentially fast - On loss (RTO), restart from $cwnd := 1\text{ MSS}$ 	

What the slide says (W5 L3, slide 2)

Recall the number guessing game: TCP congestion-control algorithms solve a similar problem. There is an **unknown** bottleneck link rate that the sender must match. If the sender sends *more* than the bottleneck rate: packet loss, delays, etc. If it sends *less*: all packets get through; successful ACKs.

Intuition: Guessing a number with only “too high” feedback

In the classic game you guess a number and hear “higher” or “lower”. TCP’s version is harsher: it hears only “fine” (ACKs) or “too much” (loss). A smart strategy: first *double* your guess until you overshoot (find the order of magnitude fast), then make *small* steps near the answer. That is precisely slow start followed by congestion avoidance.

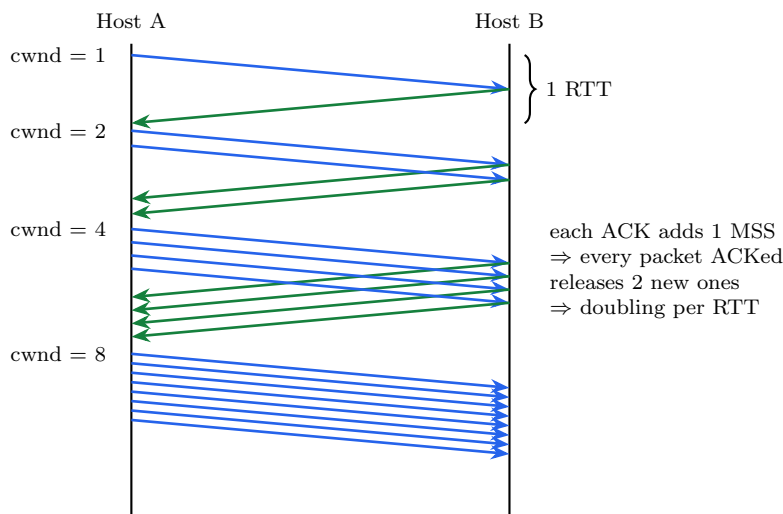
3.2 Quickly finding a rate: TCP slow start

What the slide says (W5 L3, slide 3)

Initially $\text{cwnd} = 1$ MSS (“maximum segment size”). Upon receiving an ACK of each MSS, increase cwnd by 1 MSS. Effectively, **double cwnd every RTT**. The initial rate is slow but ramps up exponentially fast. On loss (RTO), restart from $\text{cwnd} := 1$ MSS.

Definition: MSS

Maximum Segment Size: the largest payload a TCP segment carries (typically 1460 bytes on Ethernet: $1500 \text{ MTU} - 20 \text{ IP} - 20 \text{ TCP}$). cwnd is measured in bytes but usually quoted in MSS.



Formula: Why “+1 MSS per ACK” doubles per RTT

In one RTT the sender receives one ACK per segment in flight, i.e. cwnd/MSS ACKs. Each adds 1 MSS:

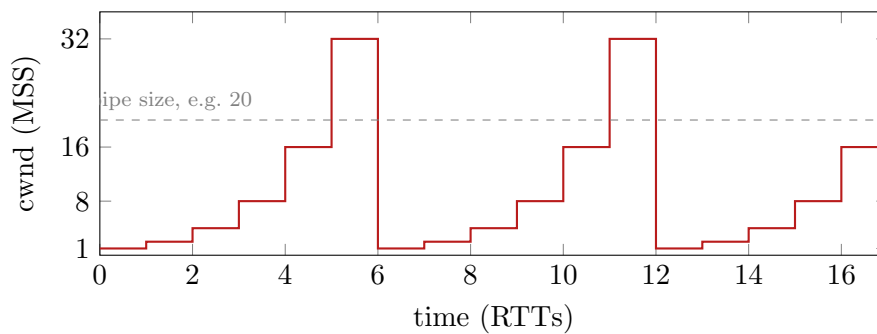
$$\text{cwnd}_{\text{next RTT}} = \text{cwnd} + \frac{\text{cwnd}}{\text{MSS}} \cdot \text{MSS} = 2 \text{cwnd}.$$

After n RTTs from 1 MSS: $\text{cwnd} = 2^n$ MSS. Time to reach a window of W MSS: $\lceil \log_2 W \rceil$ RTTs. Data sent in the first n RTTs: $1 + 2 + \dots + 2^{n-1} = 2^n - 1$ MSS.

Common pitfall: Why is it called “slow”?

It is only slow compared to the old practice of blasting a whole receiver window at once. It is exponentially *fast* growth — the name refers to its gentle *start* at 1 MSS.

3.3 Behaviour of slow start alone



What the slide says (W5 L3, slide 5 — slow start has problems)

cwnd increases too rapidly. Example: suppose the “right” window is 17: cwnd goes from 16 to 32 and then drops down to 1 $\Rightarrow$ massive packet drops. **cwnd decreases too rapidly.** Suppose the right cwnd is 31 and there is a loss when cwnd is 32: slow start resumes all the way back from cwnd 1 $\Rightarrow$ unnecessarily low throughput. Instead, perform **finer adjustments** of cwnd based on signals.

From the lecture slides: W5 L3, slides 5–6 — slow start’s problems; use slow start mainly at the beginning

Slow start has problems

- Congestion window **increases too rapidly**
 - Example: suppose the “right” window size cwnd is 17
 - cwnd would go from 16 to 32 and then dropping down to 1
 - Result: massive packet drops
- Congestion window **decreases too rapidly**
 - Suppose the right cwnd is 31, and there is a loss when cwnd is 32
 - Slow start will resume all the way back from cwnd 1
 - Result: unnecessarily low throughput
- Instead, perform **finer adjustments** of cwnd based on signals

Use slow start mainly at the beginning

- You might accelerate your car a lot when you start, but you want to make only small adjustments after.
 - Want a smooth ride, not a jerky one!
- Slow start is a good algorithm to get close to the bottleneck link rate when there is little info available about the bottleneck, e.g., starting of a connection
- **Once close enough to the bottleneck link rate**, use a different set of strategies to perform smaller adjustments to cwnd
 - Called TCP **congestion avoidance**

Intuition: Accelerating a car

You floor the accelerator from a standstill, but near cruising speed you make small corrections — you want a smooth ride, not a jerky one. Slow start is the floor-it phase: great when you know nothing about the bottleneck (start of a connection). Once close to the link rate, switch to small adjustments: **TCP congestion avoidance**.

Formula: How bad is the overshoot?

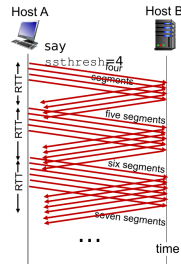
If the pipe holds P segments and slow start doubles from $2^k < P$ to $2^{k+1} > P$, up to $2^{k+1} - P$ segments (plus the bottleneck buffer’s slack) are dropped in that RTT. For $P = 17$: $32 - 17 = 15$ segments over capacity — nearly half the window.

3.4 TCP congestion avoidance: New Reno's additive increase

From the lecture slides: W5 L3, slides 8–9 — remember the last good cwnd (ssthresh); +1 MSS per RTT above it; the rules

TCP New Reno: Additive Increase

- Remember the recent past to find a good estimate of link rate
- The last good cwnd without packet drop is a good indicator
 - TCP New Reno calls this the **slow start threshold (ssthresh)**
- Increase cwnd by 1 MSS every RTT after cwnd hits ssthresh
 - Effect: increase window **additively** per RTT



TCP New Reno: Additive increase

- Start with ssthresh = 64K bytes (TCP default)
- Do slow start until ssthresh
- Once the threshold is passed, do **additive increase**
 - Add one MSS to cwnd for each cwnd worth data ACK'ed
 - For each MSS ACK'ed, $cwnd = cwnd + (MSS * MSS) / cwnd$
- Upon a TCP timeout (RTO),
 - Set cwnd = 1 MSS
 - Set ssthresh = $\max(2 * MSS, 0.5 * cwnd)$
 - i.e., the next linear increase will start at half the current cwnd

What the slide says (W5 L3, slides 8–9)

Remember the recent past to find a good estimate of the link rate: the last good cwnd without packet drop is a good indicator — TCP New Reno calls this the **slow start threshold (ssthresh)**. Increase cwnd by 1 MSS every RTT after cwnd hits ssthresh (effect: increase the window *additively* per RTT). Rules: start with ssthresh = 64 KB (TCP default). Do slow start until ssthresh. Once the threshold is passed, do additive increase: add one MSS to cwnd for each cwnd-worth of data ACKed, i.e. for each MSS ACKed, $cwnd = cwnd + (MSS \times MSS) / cwnd$. Upon a TCP timeout (RTO): set cwnd = 1 MSS; set ssthresh = $\max(2 MSS, 0.5 \times cwnd)$ — i.e. the next linear increase will start at half the current cwnd.

Definition: ssthresh

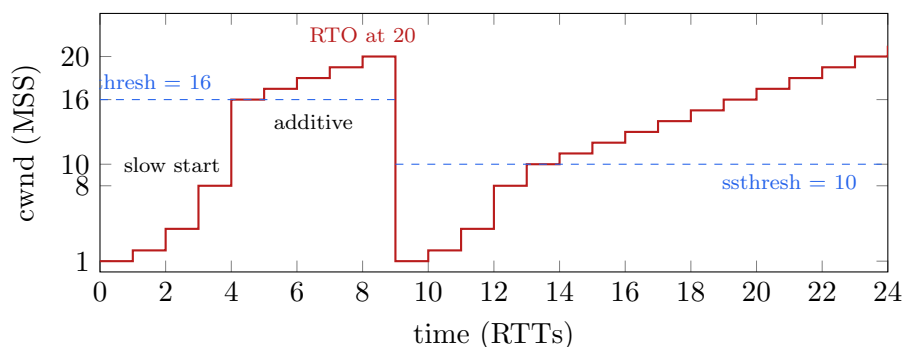
The slow-start threshold: below it, grow exponentially (slow start); at or above it, grow linearly (congestion avoidance). It stores TCP's "memory" of roughly where trouble started last time (half the window at the last loss).

Formula: Why $cwnd += MSS^2 / cwnd$ per ACK gives +1 MSS per RTT

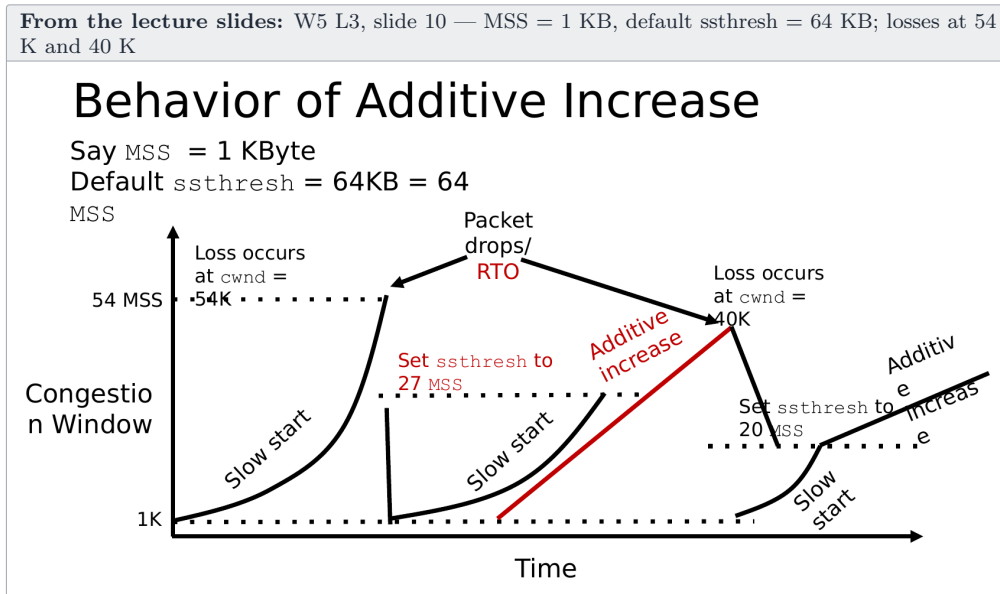
One RTT delivers cwnd/MSS ACKs, each adding $MSS^2 / cwnd$:

$$\Delta cwnd \text{ per RTT} = \frac{cwnd}{MSS} \times \frac{MSS^2}{cwnd} = MSS. \quad [\text{bytes per RTT}]$$

(Slightly less in practice, because cwnd grows during the RTT, making later increments smaller — e.g. from 10 MSS, ten ACKs give +0.96 MSS, not exactly +1.)



3.5 Behaviour of additive increase (the slide's example)



1. Start in slow start from 1 KB. Loss (RTO) occurs at cwnd = 54 K — *before* reaching the default ssthresh of 64 K.
2. On RTO: $\text{ssthresh} \leftarrow 54/2 = 27 \text{ MSS}$; $\text{cwnd} \leftarrow 1 \text{ MSS}$. Slow start again up to 27 MSS.
3. At 27 MSS switch to additive increase (the red line): +1 MSS per RTT up to 40 K, where a loss occurs.
4. On RTO: $\text{ssthresh} \leftarrow 40/2 = 20 \text{ MSS}$; $\text{cwnd} \leftarrow 1$; slow start to 20 MSS, then additive increase.

Intuition

Each loss teaches TCP something: “the pipe is somewhere below where I just was”. Halving gives a safe floor to switch to careful linear probing. Notice the slow-start phases after each RTO are short (they only climb to half the old window), and most time is spent in linear probing near the right rate.

Common pitfall: Two different reactions

In Week 5, the only loss signal is the RTO, and the reaction is drastic: $\text{cwnd} \rightarrow 1 \text{ MSS}$. Week 6 adds a gentler reaction to *triple duplicate ACKs* (fast retransmit/recovery), which halves the window instead of collapsing it.

3.6 The zoo of TCP variants

From the lecture slides: W5 L3, slide 11 — TCP congestion-control variants (feedback used, what changes, benefits, fairness)

Variant	Feedback	Required changes	Benefits	Fairness
(New) Reno	Loss	—	—	Delay
Vegas	Delay	Sender	Less loss	Proportional
High Speed	Loss	Sender	High bandwidth	
BIC	Loss	Sender	High bandwidth	
CUBIC	Loss	Sender	High bandwidth	
C2TCP ^{[11][12]}	Loss/Delay	Sender	Ultra-low latency and high bandwidth	
NATCP ^[13]	Multi-bit signal	Sender	Near Optimal Performance	
Elastic-TCP	Loss/Delay	Sender	High bandwidth/short & long-distance	
Agile-TCP	Loss	Sender	High bandwidth/short-distance	
H-TCP	Loss	Sender	High bandwidth	
FAST	Delay	Sender	High bandwidth	Proportional
Compound TCP	Loss/Delay	Sender	High bandwidth	Proportional
Westwood	Loss/Delay	Sender	Lossy links	
Jersey	Loss/Delay	Sender	Lossy links	
BBR ^[14]	Delay	Sender	BLVC, Bufferbloat	
CLAMP	Multi-bit signal	Receiver, Router	Variable-rate links	Max-min
TFRC	Loss	Sender, Receiver	No Retransmission	Minimum delay
XCP	Multi-bit signal	Sender, Receiver, Router	BLFC	Max-min
VCP	2-bit signal	Sender, Receiver, Router	BLF	Proportional
MaxNet	Multi-bit signal	Sender, Receiver, Router	BLFSC	Max-min
JetMax	Multi-bit signal	Sender, Receiver, Router	High bandwidth	Max-min
RED	Loss	Router	Reduced delay	
ECN	Single-bit signal	Sender, Receiver, Router	Reduced loss	

Variant	Feedback	Idea
(New) Reno	loss	slow start + AIMD; fast retransmit/recovery (the “textbook” TCP)
Vegas	delay	back off when RTT rises, before losses happen
CUBIC	loss	window grows as a cubic function of time since last loss (Linux default)
BBR	delay / ACK rate	model bottleneck bandwidth and min RTT; pace at that rate (Google)
Compound TCP	loss/delay	Windows’ hybrid

The key column is **Feedback**: which signal from Module 5.2 the algorithm trusts. Week 6 compares New Reno (loss) with BBR (ACK rate).

4 Formula Sheet

Advertised window	$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$
Flow-control constraint	sender's window $\leq \text{rwnd}$
Throughput bound	$\leq \text{window} / \text{RTT}$
Receiver buffer (Case 2)	$\geq W$ (sender's window); to fill the pipe $W \approx R \times \text{RTT}$
ACK spacing	$T = L / R_{\text{bottleneck}}$; ACK rate = $R_{\text{bottleneck}} / L$ packets/s
Ideal in-flight	$R_{\text{bottleneck}} \times \text{RTT}_{\text{base}}$
Slow start	$\text{cwnd} += 1 \text{ MSS per ACK} \Rightarrow \times 2 \text{ per RTT}$; $\text{cwnd} = 2^n$ after n RTTs
Additive increase	$\text{cwnd} += \text{MSS}^2 / \text{cwnd}$ per ACK $\Rightarrow +1 \text{ MSS per RTT}$
Switch point	slow start while $\text{cwnd} < \text{ssthresh}$ (default 64 KB)
On RTO	$\text{ssthresh} = \max(2 \text{ MSS}, \text{cwnd}/2)$; $\text{cwnd} = 1 \text{ MSS}$

5 Numerical Problems with Worked Solutions

Problem 1. Advertised window

A receiver's socket buffer is 64,000 bytes. So far it has received bytes up to number 120,000 (LastByteRcvd) and the app has read up to byte 100,000 (LastByteRead). What rwnd does it advertise? What if the app then reads 15,000 more bytes and 5,000 new bytes arrive?

Solution

$\text{rwnd} = 64,000 - (120,000 - 100,000) = \mathbf{44,000}$ bytes. After: $\text{LastByteRead} = 115,000$, $\text{LastByteRcvd} = 125,000 \Rightarrow \text{rwnd} = 64,000 - 10,000 = \mathbf{54,000}$ bytes.

Problem 2. Flow control caps throughput

A connection has RTT 100 ms and the receiver advertises the maximum unscaled window, 65,535 bytes. The link is 1 Gbps. What is the maximum throughput? What window would be needed to use 1 Gbps?

Solution

$65,535 \times 8 / 0.1 = \mathbf{5.24 \text{ Mbps}}$ (0.5% of the link). Needed: $R \times \text{RTT} = 10^9 \times 0.1 = 10^8$ bits = $\mathbf{12.5 \text{ MB}}$ — needs the window-scale option.

Problem 3. Sizing the receiver buffer

A sender uses a window of $W = 40$ segments of 1460 bytes. The app reads fast on average. What minimum receiver buffer does the slide rule give? If the path is 200 Mbps with RTT 30 ms, what window would fill it?

Solution

Buffer $\geq W = 40 \times 1460 = \mathbf{58,400}$ bytes. To fill the path: $2 \times 10^8 \times 0.03 = 6 \times 10^6$ bits = 750 KB $\approx \mathbf{514}$ segments — so both W and the buffer would have to be ≈ 750 KB.

Problem 4. Slow reader

The sender writes at 50 Mbps; the receiving app reads at 10 Mbps; the buffer is 5 MB. How long until the buffer is full, and what is the long-run throughput?

Solution

Net fill rate = $50 - 10 = 40$ Mbps = 5 MB/s $\Rightarrow$ full after **1 s**. After that flow control throttles the sender to the read rate: **10 Mbps**. (Slide Case 1: no buffer size prevents this for a long-lived connection.)

Problem 5. ACK clocking spacing

1500-byte packets cross a 100 Mbps access link, then a 12 Mbps bottleneck. What is the spacing T between packets after the bottleneck, the ACK spacing at the sender, and the resulting ACK-clocked sending rate?

Solution

$T = 12,000 / (12 \times 10^6) = \mathbf{1\ ms}$. ACKs arrive 1 ms apart, so the sender sends 1 packet/ms = 1000 pkt/s = **12 Mbps** — exactly the bottleneck rate, without ever measuring it.

Problem 6. Ideal steady-state window

Bottleneck 12 Mbps, base RTT 50 ms, 1500-byte packets. What in-flight amount keeps the pipe full with no queue? What happens with 80 packets in flight?

Solution

$12 \times 10^6 \times 0.05 / 12,000 = \mathbf{50}$ packets. With 80, throughput stays 12 Mbps but 30 packets sit in the bottleneck queue: extra delay = $30 \times 1\ ms = \mathbf{30\ ms}$ (RTT 80 ms instead of 50) — the “low delay” goal is violated.

Problem 7. Slow start timing

MSS = 1 KB, cwnd starts at 1 MSS, no losses. (a) After how many RTTs does cwnd reach 64 MSS? (b) How much data has been sent by then? (c) How many RTTs to deliver a 100 KB file (ignore handshake)?

Solution

(a) $2^n = 64 \Rightarrow n = \mathbf{6}$ RTTs. (b) $1 + 2 + 4 + 8 + 16 + 32 = \mathbf{63}$ KB in those 6 RTTs. (c) Cumulative after n RTTs: $2^n - 1$ KB. $2^7 - 1 = 127 \geq 100 \Rightarrow \mathbf{7}$ RTTs.

Problem 8. The slide’s “right window is 17”

The pipe (plus buffer) holds exactly 17 segments. Slow start runs from 1 MSS. When is capacity exceeded and by how much? What would additive increase from 16 do?

Solution

cwnd: 1, 2, 4, 8, 16 (fine), then **32** in the 6th RTT: $32 - 17 = \mathbf{15}$ segments over capacity $\Rightarrow$ massive drops, RTO, cwnd back to 1. Additive increase from 16: 17 (fits), 18 (1 over) — a single small loss instead of 15.

Problem 9. cwnd trace with ssthresh and an RTO

ssthresh = 16 MSS initially, cwnd starts at 1. A timeout occurs when cwnd = 20. Assume slow start stops exactly at ssthresh. List cwnd at the start of each RTT for 16 RTTs.

Solution

RTT	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
cwnd	1	2	4	8	16	17	18	19	20	1	2	4	8	10	11	12

RTTs 1–5: slow start to 16. 6–9: additive. At 20: $RTO \Rightarrow ssthresh = \max(2, 20/2) = 10$, $cwnd = 1$. RTTs 10–13: slow start ($8 \rightarrow$ capped at 10). From 14: additive.

Problem 10. *Per-ACK additive increase*

$cwnd = 10,000$ bytes, $MSS = 1000$ bytes. Compute $cwnd$ after one ACK and after ten ACKs using $cwnd += MSS^2/cwnd$.

Solution

One ACK: $10,000 + 10^6/10,000 = \mathbf{10,100}$ bytes. Ten ACKs (iterating): $\approx \mathbf{10,959}$ bytes, i.e. $+0.96 \text{ MSS} \approx$ one MSS per RTT as designed.

Problem 11. *The slide's additive-increase figure*

$MSS = 1 \text{ KB}$, default $ssthresh = 64 \text{ KB}$. A loss (RTO) occurs at $cwnd = 54 \text{ KB}$. (a) New $ssthresh$ and $cwnd$? (b) How many RTTs of slow start to reach $ssthresh$ (cap at $ssthresh$)? (c) How many RTTs of additive increase to reach 40 KB ?

Solution

(a) $ssthresh = 27 \text{ MSS}$, $cwnd = 1$. (b) 1, 2, 4, 8, 16, 27 (capped) $\Rightarrow \mathbf{5}$ RTTs. (c) $40 - 27 = \mathbf{13}$ RTTs. Then the next loss at 40 K sets $ssthresh = 20 \text{ MSS}$.

Problem 12. *Time-average of slow start*

Why is slow start fine for short transfers but bad as the only mechanism? Quantify: pipe = 100 segments, $RTT = 50 \text{ ms}$, each RTO costs 1 s of idle time, and every overshoot causes an RTO.

Solution

A cycle: 7 RTTs to climb $1 \rightarrow 64 \rightarrow 128$ (overshoot at 128) = 350 ms, then 1 s idle. Data delivered per cycle $\approx 1 + 2 + \dots + 64 = 127$ segments. Average $\approx 127/1.35 \approx 94$ segments/s vs the pipe's $100/0.05 = 2000$ segments/s — under **5%** utilisation. Remembering $ssthresh$ and probing linearly avoids repeated collapses.

6 Multiple-Choice Questions with Explanations

Q1. Flow control protects:

- (a) The network's routers
- (b) The receiver's socket buffer
- (c) The sender's CPU
- (d) The bottleneck link

(b). Congestion control protects the network.

Q2. Receiver socket buffers are used for:

- (a) Encryption and compression
- (b) Ordering and reliability

- (c) Routing and forwarding
- (d) DNS lookups

(b).

Q3. Which does *not* cause the receiver buffer to fill?

- (a) App rarely calls `recv()`
- (b) First bytes of a window lost
- (c) Heavy reordering
- (d) A very fast reading app

(d).

Q4. The advertised window is carried in:

- (a) The IP header
- (b) The TCP header's Window field in segments from the receiver
- (c) A separate UDP message
- (d) The checksum

(b).

Q5. The advertised window equals:

- (a) Total buffer size
- (b) Free space in the receiver buffer
- (c) The congestion window
- (d) The RTT

(b).

Q6. If the receiving app is slow, the sender's rate:

- (a) Increases
- (b) Decreases via a shrinking advertised window
- (c) Is unaffected
- (d) Becomes zero immediately and stays zero

(b).

Q7. Flow control matches:

- (a) Sender speed to network speed
- (b) Sender's write speed to receiver's read speed
- (c) Router speed to link speed
- (d) RTT to RTO

(b).

Q8. If the receiving app reads too slowly on a long-lived connection:

- (a) A bigger buffer fixes throughput
- (b) No buffer size prevents low throughput
- (c) Congestion control fixes it
- (d) Selective repeat fixes it

(b). Slide Case 1.

Q9. If the app reads fast on average and the sender's window is W , the receiver buffer must be at least:

- (a) $W/2$
- (b) W
- (c) $2W$
- (d) 1 MSS

(b). Slide Case 2.

Q10. $Rwnd = 64$ KB, $RTT = 50$ ms. Max throughput is about:

- (a) 1.28 Mbps
- (b) 10.5 Mbps
- (c) 64 Mbps
- (d) 1 Gbps

(b). $65,536 \times 8/0.05$.

Q11. Internet congestion control is:

- (a) Centrally controlled
- (b) A distributed algorithm at endpoints
- (c) Done only by routers
- (d) Done by DNS

(b).

Q12. Which is an implicit congestion signal measured at the sender?

- (a) ECN mark
- (b) Increasing RTT
- (c) Router configuration
- (d) The IP TTL

(b). ECN is explicit.

Q13. The “knob” in window-based congestion control is:

- (a) The advertised window
- (b) The congestion window
- (c) The TTL
- (d) The MSS

(b).

Q14. The ideal steady state aims for:

- (a) Maximum queueing
- (b) Full bottleneck utilisation with low queueing delay
- (c) Minimum sending rate
- (d) Zero ACKs

(b).

Q15. Why does “low delay” reduce to “low queueing delay”?

- (a) Propagation and transmission are treated as fixed
- (b) Queues are always empty
- (c) Propagation is zero
- (d) Transmission is zero

(a).

Q16. Packets leaving a bottleneck of rate R are spaced by:

- (a) R/L
- (b) L/R
- (c) RTT
- (d) $L \times R$

(b).

Q17. ACK clocking means:

- (a) Sending on a fixed timer
- (b) Sending a new packet when an ACK arrives
- (c) Sending only after RTO
- (d) Sending all data at once

(b).

Q18. Packet conservation says:

- (a) Never drop packets
- (b) Inject a new packet only when one leaves the network
- (c) Compress packets
- (d) Duplicate packets for safety

(b).

Q19. ACK clocking alone can be inefficient because:

- (a) ACKs are slow
- (b) It preserves, but never grows, the amount in flight
- (c) It causes loss
- (d) It ignores RTT

(b).

Q20. The grocery-store analogy for ACK clocking is:

- (a) Let everyone in at once
- (b) Let the next customer in when one exits
- (c) Close the store at noon
- (d) Let in one customer per hour

(b).

Q21. Slow start begins with cwnd =

- (a) 64 KB
- (b) 1 MSS

- (c) rwnd
- (d) ssthresh

(b).

Q22. In slow start, each ACK increases cwnd by:

- (a) $MSS^2/cwnd$
- (b) 1 MSS
- (c) Half
- (d) Nothing

(b). Hence doubling per RTT.

Q23. After 5 RTTs of slow start from 1 MSS, cwnd is:

- (a) 5 MSS
- (b) 16 MSS
- (c) 32 MSS
- (d) 64 MSS

(c). 2^5 .

Q24. A problem with slow start alone is:

- (a) It grows too slowly
- (b) It overshoots badly and collapses to 1 on loss
- (c) It needs routers
- (d) It ignores ACKs

(b).

Q25. ssthresh represents:

- (a) The receiver's buffer
- (b) The switch point from slow start to additive increase
- (c) The RTO
- (d) The MSS

(b).

Q26. Default ssthresh in the slides:

- (a) 1 MSS
- (b) 16 KB
- (c) 64 KB
- (d) 1 MB

(c).

Q27. In congestion avoidance, per ACK cwnd increases by:

- (a) 1 MSS
- (b) $MSS \times MSS/cwnd$
- (c) cwnd
- (d) 2 MSS

(b).

Q28. Congestion avoidance increases cwnd per RTT by about:

- (a) Doubling
- (b) 1 MSS
- (c) Half
- (d) 10 MSS

(b).

Q29. On an RTO, TCP New Reno sets:

- (a) $\text{cwnd} = \text{ssthresh}$
- (b) $\text{cwnd} = 1 \text{ MSS}$, $\text{ssthresh} = \max(2 \text{ MSS}, \text{cwnd}/2)$
- (c) $\text{cwnd} = \text{cwnd}/2$ only
- (d) $\text{ssthresh} = 64 \text{ KB}$

(b).

Q30. In the slide example, a loss at 54 K sets ssthresh to:

- (a) 54 MSS
- (b) 27 MSS
- (c) 20 MSS
- (d) 64 MSS

(b).

Q31. TCP Vegas primarily uses which feedback?

- (a) Loss
- (b) Delay
- (c) ECN
- (d) Advertised window

(b). From the variants table.

Q32. The final window a sender may use is:

- (a) cwnd only
- (b) rwnd only
- (c) $\min(\text{cwnd}, \text{rwnd})$
- (d) $\text{cwnd} + \text{rwnd}$

(c). Flow control and congestion control both apply (formalised in Week 6).